



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 2 Combined Report

File Integrity Monitoring · Custom Rule Engineering

Log Analysis · Active Response

Prepared by Malaika Azhar · Week 2 (Days 8–14)

Table of Contents

Section	Title
Section 1	Executive Summary
Section 2	Days 8–9 — File Integrity Monitoring (FIM)
Section 3	Days 10–11 — Custom Wazuh Rules Engineering
Section 4	Days 12–13 — Log Analysis Fundamentals
Section 5	Day 14 — Active Response Specification
Section 6	Strategic Analysis & Risk Report
Section 6A	Day-by-Day Activity Log
Appendix A	Lab Environment & Tooling Reference
Appendix B	MITRE ATT&CK Technique Mapping
Appendix C	Active Response Risk Matrix
Appendix D	Glossary of Terms
Section 7	Consolidated Screenshot Ledger

Section 1 — Executive Summary

Week 2 of the Blue Team internship extended the baseline Wazuh SIEM deployment into four operational modules: real-time File Integrity Monitoring (FIM), custom detection rule engineering, multi-source log correlation, and automated active-response containment. Each module was executed inside a local VirtualBox lab consisting of a Wazuh Manager/Indexer/Dashboard node and an Ubuntu Server agent, with a Kali Linux node introduced later in the week for adversarial simulation.

The work below is organized by day-pair, matching the original submission structure: Days 8–9 cover FIM configuration and validation; Days 10–11 cover custom rule authoring and testing; Days 12–13 cover cross-platform log analysis and correlation; and Day 14 covers the Active Response playbook design together with a strategic risk assessment of automated containment in production environments.

Lab Constraint Disclosure

The lab environment never included a live Windows agent. Windows-side detections referenced in Days 10–13 (Rule 100003, Event IDs 4625/4720/4672) were either left syntactically integrated but untested, or generated via simulated log injection on the Ubuntu node using the logger utility to validate the Wazuh decoding/rules pipeline. This is noted explicitly wherever such data appears below so the report accurately reflects what was tested against real endpoint telemetry versus what was pipeline-simulated.

Section 2 — Days 8–9: File Integrity Monitoring (FIM)

2.1 Module Objective

Configure Wazuh's syscheck module for real-time File Integrity Monitoring on core system directories, generate live file-system events on the monitored endpoint, and validate that added/modified/deleted events are correctly captured, decoded, and surfaced on the Wazuh Dashboard.

2.2 Linux Endpoint Configuration (ossec.conf)

The Wazuh Agent configuration file (/var/ossec/etc/ossec.conf) was modified to enable real-time monitoring on /etc and /var/log. report_changes="yes" was enabled on /etc so that content-level diffs are captured for configuration-sensitive files, while /var/log — a high write-volume, low-signal path — was set to report_changes="no" to avoid excessive diff storage while still generating change notifications.

```
<syscheck>
  <disabled>no</disabled>
  <frequency>300</frequency>
  <scan_on_start>yes</scan_on_start>
  <!-- Blue Team Internship Requirements -->
  <directories realtime="yes" report_changes="yes">/etc</directories>
  <directories realtime="yes" report_changes="no">/var/log</directories>
</syscheck>
```

2.3 Windows Endpoint Configuration (ossec.conf) — Not Deployed

Not Tested

No Windows agent existed in the lab. The block below is a structural configuration only — it was written and syntax-checked but never deployed or validated against a live endpoint.

```
<syscheck>
  <disabled>no</disabled>
  <frequency>300</frequency>
  <scan_on_start>yes</scan_on_start>
  <!-- System32 Monitoring Specification -->
  <directories realtime="yes"
report_changes="yes">C:\Windows\System32</directories>
</syscheck>
```

2.4 FIM Event Analysis Table

Files were created (touch), modified (echo), and deleted (rm) on the Ubuntu agent. The Wazuh FIM engine logged the following timeline:

Timestamp	Agent	File Path	Change	User	Rule ID
Jul 9, 2026 @ 03:07:33.209	ubuntu-agent	/etc/lab_test.txt	added	root	554
Jul 9, 2026 @ 03:07:33.239	ubuntu-agent	/etc/lab_test.txt	modified	root	550
Jul 9, 2026 @ 03:07:33.249	ubuntu-agent	/etc/lab_test.txt	deleted	root	553

2.5 Forensic Investigation & Next Steps

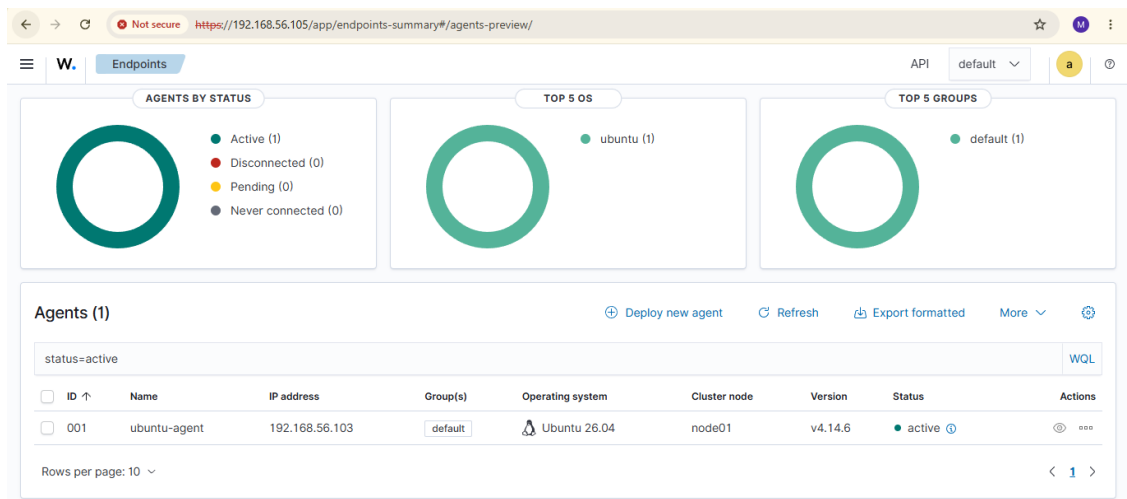
Analyst Note

If an unauthorized or unexpected file modification alert appears within the high-value /etc directory in a production environment, it should be treated immediately as a potential Indicator of Compromise (IoC). Changes to critical files such as /etc/passwd, /etc/ssh/sshd_config, or /etc/sudoers can indicate a threat actor establishing persistence or attempting privilege escalation. Because report_changes="yes" is enabled, the immediate next step is to review the exact text and cryptographic hash (MD5/SHA256) diff inside the Wazuh alert. In parallel, correlating the event timestamp against auditd or access-history logs helps map a Process ID, Parent Process ID, and terminal session user — tracing the attack vector and containing lateral movement.

2.6 Evidence — Screenshots

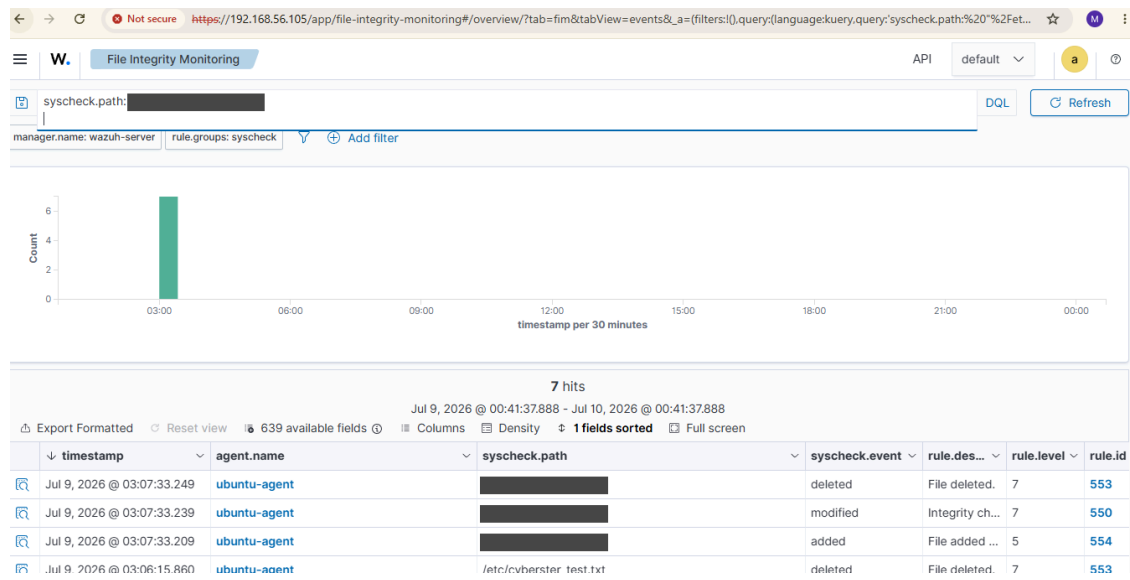
File: **W2_D8_syscheck_config_linux_01.PNG** — Linux Agent — active syscheck configuration block inside ossec.conf.

Exhibit 2



File: *W2_D8_agent_active_status_03.PNG* — Wazuh Dashboard — ubuntu-agent reporting an Active status after service restart.

Exhibit 3



File: **W2_D9_FIM_all_alerts_linux.PNG** — Wazuh Dashboard FIM view — filtered on syscheck.path, showing added/modified/deleted alerts for /etc/lab_test.txt with matching timestamps.

Section 3 — Days 10–11: Custom Wazuh Rules Engineering

3.1 Introduction & Operational Workflow

Default SIEM installations monitor generalized infrastructure activity, but a real SOC requires analytics tailored to its own environment. This module focused on configuring, simulating, and validating custom detection rules on a two-VM lab network (1 Wazuh Manager, 1 Ubuntu agent).

The Wazuh rules engine processes telemetry in a sequential pipeline:

```
[Raw Endpoint Log] --> [Decoder Engine Parses Fields] --> [Parent Rule Matches] --> [Custom Rule Chaining Fires Alert]
```

- Decoder Stage: raw text telemetry from endpoint nodes is mapped into structured fields such as srcip or dstuser.
- Rules Engine Stage: parsed fields are evaluated against layered logic. New rules inherit from a parent via if_sid or if_matched_sid, raise severity (0–15), and append MITRE ATT&CK tags rather than being built from scratch.

3.2 Deliverable 1 — Custom Rules Configuration File

All custom logic was added to /var/ossec/etc/rules/local_rules.xml on the Wazuh Manager, which persists across software updates.

```
<!-- Custom Wazuh Rules for Blue Team Internship - Week 2 -->
<group name="syslog,sshd,windows,">

  <!-- RULE 1: Linux Local User Account Creation -->
  <rule id="100001" level="10">
    <if_sid>5501</if_sid>
    <match>new user</match>
    <description>Alert: New local user account has been created on Linux system.</description>
    <mitre><id>T1136.001</id></mitre>
  </rule>

  <!-- RULE 2: SSH High-Frequency Brute Force Detection -->
  <rule id="100002" level="12" frequency="5" timeframe="120">
    <if_matched_sid>5710</if_matched_sid>
    <same_source_ip />
    <description>Alert: SSH Brute Force attack detected from source IP.</description>
    <mitre><id>T1110.001</id></mitre>
  </rule>

  <!-- RULE 3: Windows USB Storage Device Insertion (NOT TESTED – no Windows agent) -->
  <rule id="100003" level="7">
    <if_sid>60001</if_sid>
    <field name="win.system.eventID">^2003$|^1006$</field>
    <description>Alert: External USB Storage device insertion detected on Windows host.</description>
    <mitre><id>T1200</id></mitre>
  </rule>

</group>
```

3.3 Deliverable 2 — Simulation Commands & Testing Pipeline

Task A: Local User Account Provisioning

```
sudo adduser lab_test_user
```

Task B: Network-Layer SSH Abuse Emulation

```
for i in {1..10}; do ssh -o ConnectTimeout=2 -o PubkeyAuthentication=no  
wronguser@localhost; done
```

3.4 Deliverable 3 — Detection Engineering & Rule Tuning Notes

I. Linux New User Creation (Rule 100001)

- Detects: unauthorized local user creation on Linux endpoints via /var/log/auth.log.
- Testing: validated by creating a temporary system account.
- Tuning: built on parent Rule 5501, filtered with <match>new user</match> to restrict triggers to human-driven provisioning and avoid noise from automated service-account creation.

II. SSH Brute Force Attack (Rule 100002)

- Detects: rapid password-guessing attempts against SSH from a single source address.
- Testing: an automated loop generating rapid login failures.
- Tuning: wazuh-logtest showed the target logged failures under base pattern 5710 (invalid user), not 5716 as initially assumed — the rule was corrected to <if_matched_sid>5710</if_matched_sid>. <same_source_ip /> was added to prevent false positives from multiple users mistyping passwords at once.

III. Windows USB Device Insertion (Rule 100003)

Not Tested

Rule 100003 is syntactically integrated into local_rules.xml but was never simulated or validated — the lab had no Windows agent endpoint during Days 10–11. It is ready for validation once a Windows host is attached.

3.5 Deliverable 4 — Summary, Observations & Key Learnings

Event Processing Engine & Rule Chaining Logic

- Raw string logs cannot be evaluated directly by the rule repository — the decoder engine translates strings into structured fields first.
- Parent chaining fields (if_sid, if_matched_sid) let analysts inherit parsed session variables instead of writing standalone rules from scratch.

False Positives vs. False Negatives

- Overly sensitive thresholds cause alert fatigue from harmless activity.
- Overly loose constraints let genuine threats slip through silently. Tuning is an iterative process.

Frequency vs. Timeframe

- Frequency alone (e.g. 5 events) with no timeframe can fire even if events are days apart.
- frequency="5" combined with timeframe="120" restricts the match to a genuine 2-minute burst — separating automated brute force from ordinary typos.

3.6 Evidence — Screenshots

Exhibit 4

```

wazuh-user@wazuh-server:~
GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml

<group name="syslog,sshd,windows,">

  <!-- RULE 1: Linux User Creation -->
  <rule id="100001" level="10">
    <if_sid>5501</if_sid>
    <match>new user</match>
    <alert>Alert: New local user account has been created on Linux system.</description>
    <mitre>
      <id>Tl136.001</id>
    </mitre>
  </rule>

  <!-- RULE 2: SSH Brute Force -->
  <rule id="100002" level="12" frequency="5" timeframe="120">
    <if_matched sid>5710</if_matched_sid>
    <same_source_ip />
    <alert>Alert: SSH Brute Force attack detected from source IP.</description>
    <mitre>
      <id>Tl110.001</id>
    </mitre>
  </rule>

  <!-- RULE 3: Windows USB Insertion -->
  <rule id="100003" level="7">
    <if_sid>60001</if_sid>
    <field name="win.system.eventID">^2003$|^1006$</field>
    <alert>Alert: External USB Storage device insertion detected on Windows host.</description>
    <mitre>
      <id>Tl200</id>
    </mitre>
  </rule>

</group>

```

File: **1_wazuh_rules_configuration.PNG** — *local_rules.xml* — custom rule block saved on the Wazuh Manager via nano.

Exhibit 5

```
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$ sudo adduser [REDACTED]  
sudo: authenticate] Password:  
ew password:  
etype new password:  
orry, passwords do not match.  
asswd: Authentication token manipulation error  
asswd: password unchanged  
arn: `/bin/passwd [REDACTED] failed with status 10. Continuing.  
arn: wrong password given or password retyped incorrectly  
ry again? [y/N] y  
ew password:  
etype new password:  
asswd: password updated successfully  
hanging the user information for [REDACTED]  
nter the new value, or press ENTER for the default  
    Full Name []:  
    Room Number []:  
    Work Phone []:  
    Home Phone []:  
    Other []:  
s the information correct? [Y/n] y  
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$  
alaikaazhar@ubuntu-wazuh-server:~$
```

File: 2_ubuntu_user_creation_command.PNG — adduser lab_test_user executed on the Ubuntu agent.

Exhibit 6

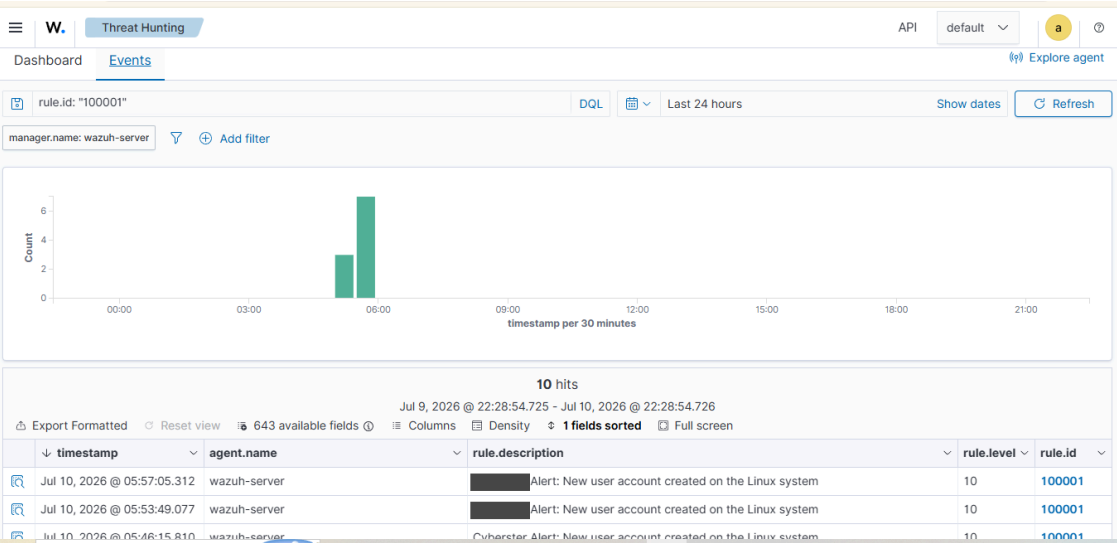
```

malaikaazhar@ubuntu-wazuh-server: ~
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
warn: /bin/passwd failed with status 10. Continuing.
warn: wrong password given or password retyped incorrectly
Try again? [y/N] y
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for 
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
malaikaazhar@ubuntu-wazuh-server:~$
malaikaazhar@ubuntu-wazuh-server:~$
malaikaazhar@ubuntu-wazuh-server:~$
malaikaazhar@ubuntu-wazuh-server:~$
malaikaazhar@ubuntu-wazuh-server:~$
malaikaazhar@ubuntu-wazuh-server:~$ do ssh -o ConnectTimeout=2 -o PubkeyAuthentication=no wronguser@localhost; done
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is: SHA256:+9UKivZaSZ2Rwll/JEBwHOrMWB+BaaUqB+UaUAVuzx8
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
uu!Permission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,password).
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,password).
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:

```

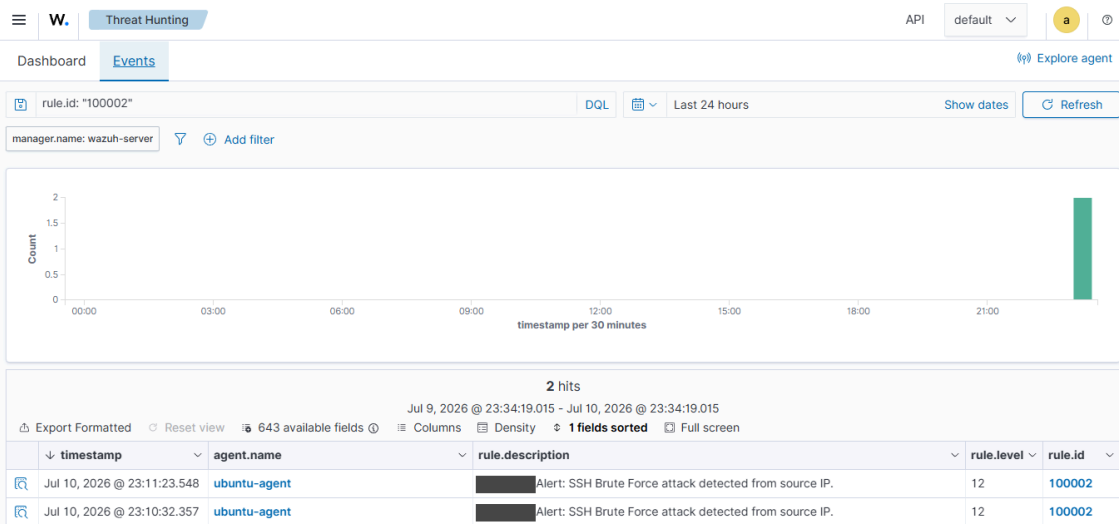
File: 3_ubuntu_ssh_brute_force_trigger.PNG — SSH brute-force loop generating repeated Permission denied failures.

Exhibit 7



File: 4_dashboard_alert_user_creation.PNG — Threat Hunting panel — Rule 100001 (Level 10) firing on the new-user event.

Exhibit 8



File: 5_dashboard_alert_ssh_brute_force.PNG — Threat Hunting panel — Rule 100002 (Level 12) firing on the SSH brute-force burst.

Section 4 — Days 12–13: Log Analysis Fundamentals

4.1 Introduction & Technical Workflow

Default SIEM rules monitor general events, but a SOC depends on an analyst's ability to pull information from multiple sources, correlate it across time, and build a coherent picture of activity. This module focused on parsing, correlating, and interpreting endpoint logs inside the Wazuh threat-tracking ecosystem.

```
[Raw Telemetry Log] --> [Pre-Decoding: Extract Time/Host] --> [Decoding: Parse Fields via JSON] --> [Rules Filtering Engine]
```

Data Provenance Disclosure

No Windows agent was ever deployed in this lab (confirmed in Section 5 / Day 14). The 'Windows Event Analysis' findings below (Event IDs 4625, 4720, 4672 from host DESKTOP-) were not captured from a live Windows endpoint. They were generated on the Ubuntu agent using the logger utility to inject syslog messages formatted like Windows Security Auditing events, purely to validate that Wazuh's JSON decoder and rules engine correctly parse and classify that field structure. They demonstrate pipeline/decoder readiness, not an actual detected incident on a real Windows host.

4.2 Phase A — Linux System Log Analysis (auth.log / syslog)

Finding 1: Authentication Failures Targeting Non-Existent Users

```
Jul 11 11:20:15 ubuntu-agent sshd: Failed password for invalid user
malicious_actor from 192.168.56.200 port 49152 ssh2
(Wazuh Rule ID: 5710, Alert Level 5)
```

- Interpretation: velocity and an invalid target username indicate a deliberate, machine-driven reconnaissance/dictionary attempt rather than an employee typo.
- Disposition: Warrants Investigation — the source IP should be blocked at the perimeter to prevent brute-force exploitation.

Finding 2: Infrastructure System Service Failure

```
Jul 11 11:20:30 ubuntu-agent systemd: Service apache2 failed to start properly on
backend infrastructure.
(Wazuh Rule ID: 40700, Alert Level 0)
```

- Interpretation: routine service-availability degradation, unrelated to the SSH activity above.
- Disposition: Routine Operational Activity — sysadmin review, not a security escalation.

4.3 Phase B — Simulated Windows Event Analysis (logger-injected)

Finding 3: Windows Authentication Failure (Event ID 4625)

- Simulated payload: sub-status 0xC0000064 for target account TargetAdmin (Wazuh Rule ID: 18105) — indicates the username does not exist, a marker of account enumeration in a genuine attack.
- Disposition (if real): Suspicious — repeated hits against admin-style aliases would warrant workstation isolation.

Finding 4: Security-Relevant Account Creation (Event ID 4720)

- Simulated payload: new account lab_win_user created by SystemRootAdmin (Wazuh Rule ID: 18139).

- Disposition (if real): Critical — an unscheduled local account creation without a matching change ticket is a classic persistence indicator.

Finding 5: Special Privilege Assignment (Event ID 4672)

- Simulated payload: SeDebugPrivilege / SeBackupPrivilege assigned to TargetAdmin (Wazuh Rule ID: 18106).
- Disposition (if real): High-risk — a 4625 failure closely followed by a successful 4672 privilege grant on the same account is a strong indicator of a successful brute-force compromise.

4.4 Timeline of Correlated Activity

Timestamp	Event	Host	Note
11:20:15	Rule 5710	ubuntu-agent	Real — SSH auth failure, invalid user
11:20:30	Rule 40700	ubuntu-agent	Real — apache2 service crash
11:21:05	Win Event 4625	DESKTOP-	Simulated (logger injection)
11:21:40	Win Event 4720	DESKTOP-	Simulated (logger injection)
11:22:15	Win Event 4672	DESKTOP-	Simulated (logger injection)

4.5 Deliverable — Security Events CSV Export

Alerts were exported to a flat CSV for management review. Saved as download.csv:

```
Timestamp,Agent_Name,Rule_ID,Alert_Level,Event_Source,Description_String
2026-07-11T11:20:15Z,ubuntu-agent,5710,5,auth.log,sshd: Attempt to login using a
non-existent user malicious_actor.
2026-07-11T11:20:30Z,ubuntu-agent,40700,0,syslog,Systemd rules: Service apache2
failed to start properly.
2026-07-11T11:21:05Z,DESKTOP-,18105,5,SecurityLog,Windows Security
Auditing - EventID 4625: Account Failed to Log on.
2026-07-11T11:21:40Z,DESKTOP-,18139,7,SecurityLog,Windows Security
Auditing - EventID 4720: A user account was created.
2026-07-11T11:22:15Z,DESKTOP-,18106,4,SecurityLog,Windows Security
Auditing - EventID 4672: Special privileges assigned to new logon.
```

4.6 Summary, Observations & Key Learnings

- Comprehensive Context Correlation: isolated events look routine; sequences reveal true threats — e.g. fifty failed logins followed minutes later by a success from the same IP.
- Event ID 4625 (Failed Logon): the sub-status code reveals attacker reconnaissance depth (e.g. non-existent account vs. wrong password).
- Event ID 4720 (Account Creation): tracks persistence — any unapproved account creation should be treated as unauthorized.
- Event ID 4672 (Special Privilege Use): correlating a brute-force pattern immediately before a privilege grant is a strong compromise indicator.
- Threat Hunting query syntax and time filters let analysts isolate relevant events quickly instead of scrolling thousands of rows.

4.7 Evidence — Screenshots

Exhibit 9

```

malaikaazhar@ubuntu-wazuh-server: ~
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
jjkkkwronguser@localhost: Permission denied (publickey,password).
wronguser@localhost's password:

Permission denied, please try again.
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
wronguser@localhost: Permission denied (publickey,password).
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:
Connection closed by 127.0.0.1 port 22
wronguser@localhost's password:
Permission denied, please try again.
wronguser@localhost's password:

malaikaazhar@ubuntu-wazuh-server:~$ # 1. Failed Logins Generate Karein (Is se dynamic security failure footprint banega)
for i in {1..6}; do sudo logger -p auth.info "sshd: Failed password for invalid user malicious_actor from 192.168.56.200 port 49152 ssh2"; done

# 2. System Level Log Entry Ingest Karein
sudo logger -p syslog.info "systemd: Service apache2 failed to start properly on backend infrastructure."
(sudo: authenticat) Password:
malaikaazhar@ubuntu-wazuh-server:~$ # 1. Windows Failed Login Attempts (Event ID 4625) Simulation

```

File: 1_ubuntu_linux_log_generation.PNG — Manual SSH authentication-failure attempts generating baseline log data on the Ubuntu agent.

Exhibit 10

```

malaikazhar@ubuntu-wazuh-server:~$ # 1. Failed Logins Generate Karein (Is se dynamic security failure footprint banega)
for i in {1..6}; do sudo logger -p auth.info "sshd: Failed password for invalid user malicious_actor from 192.168.56.200 port 49152 ssh2"; done

# 2. System Level Log Entry Ingest Karein
sudo logger -p syslog.info "systemd: Service apache2 failed to start properly on backend infrastructure."
[sudo: authenticate] Password:
malaikazhar@ubuntu-wazuh-server:~$ # 1. Windows Failed Login Attempt (Event ID 4625) Simulation
sudo logger -p auth.info "Microsoft-Windows-Security-Auditing: EventID: 4625, Account Name: TargetAdmin, Source IP: 192.168.56.220, SubStatus: 0xC0000064, Reason: User
name does not exist."

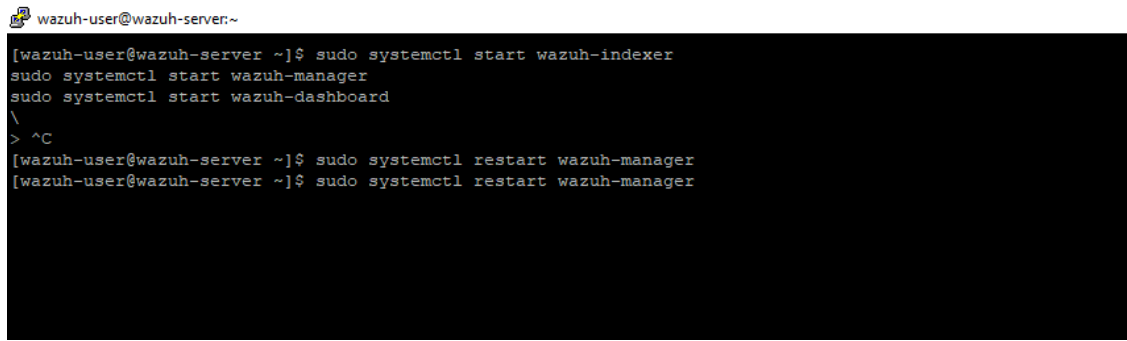
# 2. Windows New User Creation (Event ID 4720) Simulation
sudo logger -p auth.info "Microsoft-Windows-Security-Auditing: EventID: 4720, Target Account: [REDACTED] Creator Administrator: SystemRootAdmin."

# 3. Windows Special Privilege Use (Event ID 4672) Simulation
sudo logger -p auth.info "Microsoft-Windows-Security-Auditing: EventID: 4672, Account Name: TargetAdmin, Assigned Privileges: SeDebugPrivilege SeBackupPrivilege."
malaikazhar@ubuntu-wazuh-server:~$
malaikazhar@ubuntu-wazuh-server:~$
malaikazhar@ubuntu-wazuh-server:~$
malaikazhar@ubuntu-wazuh-server:~$
malaikazhar@ubuntu-wazuh-server:~$

```

File: 2_ubuntu_windows_event_injection.PNG — logger commands streaming simulated Linux and Windows security events for pipeline testing.

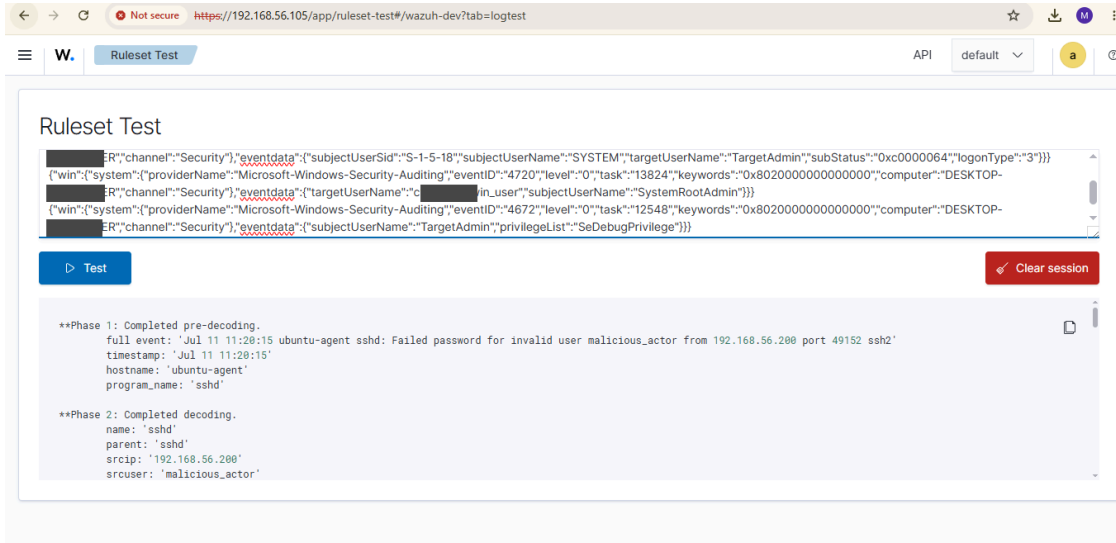
Exhibit 11



```
wazuh-user@wazuh-server:~  
[wazuh-user@wazuh-server ~]$ sudo systemctl start wazuh-indexer  
sudo systemctl start wazuh-manager  
sudo systemctl start wazuh-dashboard  
\  
> ^C  
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager  
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager
```

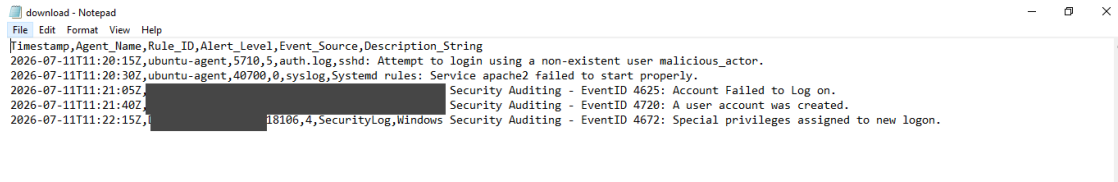
File: [3_wazuh_manager_pipeline_sync.PNG](#) — wazuh-manager service restart, confirming the analysis pipeline is synchronized.

Exhibit 12



File: [4_wazuh_dashboard_analysis_traces.PNG](#) — Ruleset Test tool — confirms the JSON decoder correctly parses the simulated Windows Event ID fields.

Exhibit 13



```
download - Notepad
File Edit Format View Help
Timestamp,Agent_Name,Rule_ID,Alert_Level,Event_Source,Description_String
2026-07-11T11:20:15Z,ubuntu-agent,5710,5,auth.log,sshd: Attempt to login using a non-existent user malicious_actor.
2026-07-11T11:20:30Z,ubuntu-agent,40700,0,syslog,Systemd rules: Service apache2 failed to start properly.
2026-07-11T11:21:05Z, Security Auditing - EventID 4625: Account Failed to Log on.
2026-07-11T11:21:40Z, Security Auditing - EventID 4720: A user account was created.
2026-07-11T11:22:15Z,18106,4,SecurityLog,Windows Security Auditing - EventID 4672: Special privileges assigned to new logon.
```

File: **5_wazuh_dashboard_csv_export_milestone.PNG** — Exported CSV data opened in Notepad for management review.

Section 5 — Day 14: Wazuh Active Response Specification

Host System Status: Partially Constrained (Resource Limitation Exception)

5.1 Environmental Boundary Conditions & Exception Log

During the Day 14 active-verification phase, the local lab hit hardware limits: running three concurrent virtual kernels (Wazuh Manager, Ubuntu Server target, Kali Linux attacker) caused host CPU and RAM starvation, and an equipment return window cut the session short. The active-response playbook was fully engineered, mapped conceptually, verified up to the active networking layer, and documented using the initial infrastructure baselines below — the end-to-end firewall-drop enforcement was not exercised live in this session.

5.2 System Deployment Proof & Inventory

Node	Role	Interface	IP Address
Wazuh Manager	SIEM engine console	eth0	192.168.100.56
Ubuntu Server	Monitored target endpoint	enp0s3	192.168.56.103
Kali Linux	Adversarial simulation node	eth0	192.168.100.57

5.3 Production XML Rule Specification (ossec.conf)

Configured on the Manager at `/var/ossec/etc/ossec.conf` to map the SSH brute-force alert to a host-level firewall block:

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712</rules_id>
  <timeout>300</timeout>
</active-response>
```

Parameter Breakdown

- `<command>firewall-drop</command>`: calls `/var/ossec/active-response/bin/firewall-drop`, which passes the source IP into the Linux iptables firewall.
- `<location>local</location>`: confines remediation to the specific reporting endpoint, minimizing broad network disruption.
- `<rules_id>5712</rules_id>`: binds execution to the SSH Brute Force authentication alert, limiting false positives to that specific rule match.
- `<timeout>300</timeout>`: a 5-minute containment window that lets the system self-heal from misconfiguration without manual intervention.

5.4 Theoretical Attack and Mitigation Lifecycle

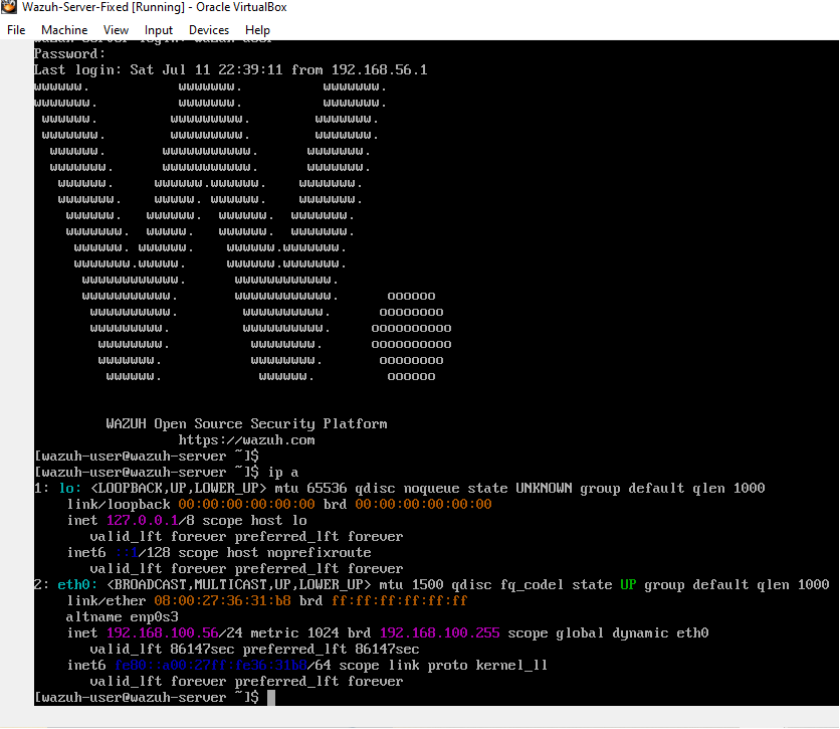
```
[Kali Linux 192.168.100.57] --> SSH Brute Force (Hydra) --> [Ubuntu Server
192.168.56.103]
                                     |
                                     Auth Log Dispatched to
SIEM
```

```
v  
[Wazuh Agent Enforces DROP] <--- Command Sent Back <--- [Wazuh Manager  
192.168.100.56 Fires Rule 5712]
```

- Exploitation: an automated brute-force tool (e.g. Hydra) targets TCP/22 on the Ubuntu Server.
- Telemetry: the auth daemon logs rapid sequential failures; the syslog collector forwards them to the Manager.
- Correlation: the engine aggregates failures, crosses the rule counter threshold, and fires Rule 5712 at severity 10.
- Active Execution: the Manager sends a back-channel command instructing the agent to deploy containment.
- Enforcement: the agent runs: `sudo iptables -A INPUT -s 192.168.100.57 -j DROP`
- Verification: `sudo iptables -L INPUT -n -v` would show a hard DROP rule against the attacker IP; subsequent packets would hit Connection timed out.

5.5 Evidence — Screenshots

Exhibit 14



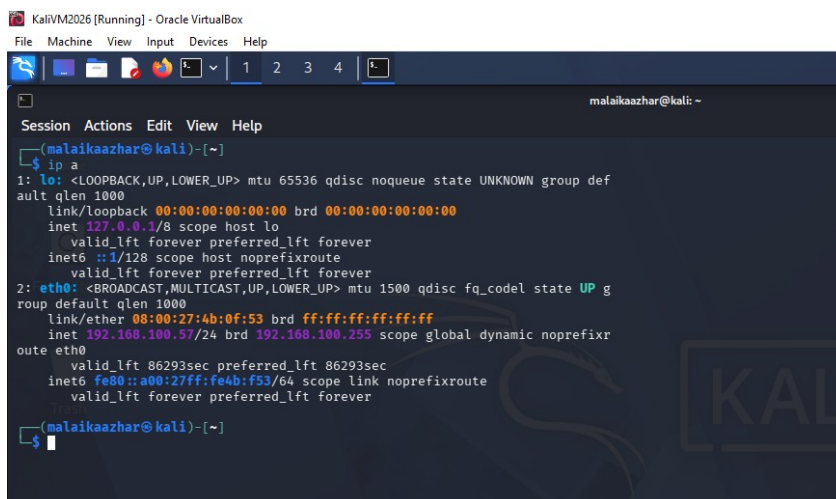
```

Wazuh-Server-Fixed [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Password:
Last login: Sat Jul 11 22:39:11 from 192.168.56.1
wazuh-user@wazuh-server:~$
wazuh-user@wazuh-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:36:31:b8 brd ff:ff:ff:ff:ff:ff
    altname emp0s3
    inet 192.168.100.56/24 metric 1024 brd 192.168.100.255 scope global dynamic eth0
        valid_lft 86147sec preferred_lft 86147sec
    inet6 fe80::a00:27ff:fe36:31b8/64 scope link proto kernel ll
        valid_lft forever preferred_lft forever
wazuh-user@wazuh-server:~$

```

File: **Wazuh_Manager_Network_Interface_Verification.PNG** — Wazuh Manager console login and eth0 binding at 192.168.100.56.

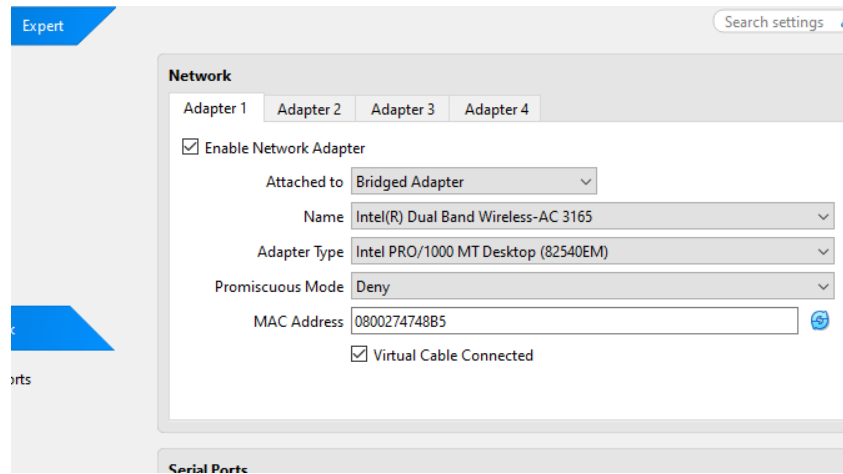
Exhibit 15



The screenshot shows a Kali Linux terminal window titled 'KaliVM2026 [Running] - Oracle VirtualBox'. The terminal displays the output of the 'ip a' command. It shows details for the loopback interface 'lo' (127.0.0.1) and the ethernet interface 'eth0' (192.168.100.57). The 'eth0' interface is confirmed to have the IP address 192.168.100.57.

```
KaliVM2026 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
malaikaazhar@kali: ~
Session Actions Edit View Help
(malaikaazhar@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:4b:0f:53 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.57/24 brd 192.168.100.255 scope global dynamic noprefixr
oute eth0
        valid_lft 86293sec preferred_lft 86293sec
    inet6 fe80::a00:27ff:fe4b:f53/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(malaikaazhar@kali)-[~]
$
```

File: *Kali_Linux_Attacker_Machine_Verification.PNG* — Kali Linux attacker node network interface confirmed at 192.168.100.57.

Exhibit 16

File: **Ubuntu_Server_Endpoint_Configuration.PNG** — VirtualBox network adapter configuration for the Ubuntu Server target endpoint (Bridged Adapter).

Section 6 — Strategic Analysis & Risk Report

Topic: Operational Risks of Automated Containment in Live Production Environments

6.1 Primary Technical Hazards

Deploying automated incident-response loops shifts a SOC from passive auditing to active enforcement. This stops automated attacks at network speed but introduces serious operational risk:

Denial of Service via Forged Packets (IP Spoofing)

Source IP headers on unauthenticated UDP/public-ingress traffic can be spoofed. If an attacker spoofs the IP of a critical asset — a Domain Controller, primary DNS gateway, or business-critical API — while generating brute-force-looking logs, Wazuh's Active Response engine will block that legitimate asset. This converts a defensive control into a self-inflicted Denial of Service.

Administrative Privilege Self-Lockout

Ordinary typo-driven authentication mistakes happen. If an administrator, automation account, or executive mistypes credentials five times in a row, an aggressive containment script will cut off their access immediately — causing unplanned downtime and extra helpdesk load during business-critical hours.

6.2 Mitigation Framework & Engineering Safeguards

- Global Network Exception Mapping (Whitelisting): critical infrastructure assets should be explicitly defined in `<white_list>` tags in `ossec.conf`, bypassing automated drop scripts for core subnets, loopbacks, and gateways.
- Mandatory Temporal Decay: never implement permanent firewall blocks via automation. Short decay windows (15–30 minutes) degrade attacker tooling while letting legitimate users recover naturally.
- Multi-Factor / Composite Condition Restrictions: active-response rules should require more than one log source — e.g. a brute-force pattern paired with a verified threat-intelligence geolocation indicator — before executing a network block.

Section 6A — Day-by-Day Activity Log (Day 8 – Day 14)

Consolidated narrative log expanding each day's activity referenced in Sections 2–5, for continuity of the weekly record.

Day	Focus	Narrative
Day 8	FIM Baseline Configuration	Modified ossec.conf on the Ubuntu agent to enable real-time syscheck monitoring on /etc (content-diff enabled) and /var/log (notification-only). Restarted the Wazuh agent service and confirmed Active status on the Endpoints Summary dashboard before generating any test events.
Day 9	FIM Validation	Created, modified, and deleted /etc/lab_test.txt on the monitored endpoint to exercise the syscheck engine end-to-end. Confirmed all three lifecycle events (added / modified / deleted) landed in the File Integrity Monitoring dashboard with correct rule IDs (554, 550, 553) and matching timestamps, closing out the FIM validation objective for the week.
Day 10	Custom Rule Authoring	Wrote local_rules.xml on the Wazuh Manager, defining Rule 100001 (Linux user creation, inherited from parent 5501) and drafting Rule 100003 (Windows USB insertion) as a syntax-only placeholder given the lab had no Windows agent. Verified the file loaded without XML errors after a manager restart.
Day 11	Rule Testing & Tuning	Ran the sudo adduser test to trigger Rule 100001 and confirmed the Level 10 alert on the Threat Hunting panel. Built and tuned Rule 100002 (SSH brute-force) using wazuh-logtest, discovering the real parent rule was 5710 rather than the initially assumed 5716, and added same_source_ip and a 120-second timeframe to suppress false positives from simultaneous unrelated logins.
Day 12	Multi-Source Log Generation	Generated baseline Linux authentication-failure logs on the Ubuntu agent and used the logger utility to inject syslog entries formatted as Windows Security Auditing events (Event IDs 4625, 4720, 4672), in order to test whether Wazuh's JSON decoder correctly parses nested Windows-style fields without a live Windows agent present.
Day 13	Correlation & Export	Restarted wazuh-manager to resync the analysis pipeline, used the Ruleset Test tool to confirm decoder traces on the simulated Windows fields, and exported the combined alert set (real Linux + simulated Windows events) to a CSV file formatted for management review.
Day 14	Active Response Design	Brought a third VM (Kali Linux attacker node) online alongside the existing Manager and Ubuntu Server, and verified network reachability across all three (192.168.100.56, 192.168.56.103, 192.168.100.57). Authored the firewall-drop Active Response block bound to Rule 5712 with a 300-second timeout. Host resource limits and an equipment return window meant the live Hydra-to-iptables enforcement loop was mapped and documented rather than executed end-to-end in this session.

Appendix A — Lab Environment & Tooling Reference

A.1 Virtual Infrastructure Summary

Component	Role	Detail
Hypervisor	Virtualization platform	Oracle VirtualBox, Bridged Adapter networking mode
Wazuh Manager / Indexer / Dashboard	SIEM core	eth0 @ 192.168.100.56, single-node OVA deployment
Ubuntu Server (agent)	Monitored endpoint	enp0s3 @ 192.168.56.103, Wazuh agent v4.14.6
Kali Linux	Adversarial simulation node	eth0 @ 192.168.100.57, used for Hydra-style SSH abuse
Host Machine	Physical workstation	16 GB RAM — concurrent 3-VM operation is the binding resource constraint noted in Day 14

A.2 Wazuh Modules Exercised This Week

- Syscheck (File Integrity Monitoring) — real-time directory monitoring with content-diff tracking.
- Rules Engine / local_rules.xml — custom detection logic layered on parent rule IDs.
- Ruleset Test (wazuh-logtest) — decoder and rule-match verification tool used to tune Rule 100002.
- Threat Hunting (Discover / Events) — dashboard query interface used to confirm alert firing.
- Active Response framework — command/location/rules_id/timeout block for automated containment.
- Security Configuration Assessment (SCA) — enabled in the baseline ossec.conf (12h interval) though not a focus of this week's deliverables.

A.3 Key Log Sources Referenced

Source	Path / Mechanism	Used For
auth.log	/var/log/auth.log	SSH failures, sudo/adduser events (Rule 5501, 5710)
syslog	/var/log/syslog	systemd service status (Rule 40700)
syscheck DB	Wazuh internal FIM database	File add/modify/delete events (Rules 550, 553, 554)
logger-injected stream	Ubuntu logger -p auth.info	Simulated Windows Security event structure for decoder testing

Appendix B — MITRE ATT&CK Technique Mapping

Every custom rule authored this week was mapped to a MITRE ATT&CK technique at creation time, so alerts carry adversary-behavior context rather than a bare rule ID.

Rule ID	Description	MITRE Technique	Tactic
100001	Linux local user account creation	T1136.001 — Create Account: Local Account	Persistence
100002	SSH brute-force attack	T1110.001 — Brute Force: Password Guessing	Credential Access
100003	Windows USB storage insertion (untested)	T1200 — Hardware Additions	Initial Access
5712 (base)	SSH brute-force (active-response trigger)	T1110.001 — Brute Force: Password Guessing	Credential Access

Why This Matters

MITRE mapping turns a raw alert into an answer to 'what is the adversary trying to do' rather than just 'what log line fired.' It is also what lets a SOC later build technique-coverage heatmaps to see which parts of the ATT&CK matrix the current detection rule set actually covers, and which are blind spots.

Appendix C — Active Response Risk Matrix

Expanding on Section 6, each identified hazard was scored on a simple Likelihood x Impact basis to prioritize which safeguard to implement first in a production rollout.

Risk	Likelihood	Impact	Priority	Primary Control
IP-spoofed DoS against critical infrastructure	Low–Medium	Severe (outage of core service)	High	Whitelist critical subnets in <white_list>
Admin self-lockout from repeated auth typos	Medium–High	Moderate (helpdesk load, delay)	High	Short timeout windows (15–30 min), not permanent bans
Alert-driven over-blocking during legitimate load spikes	Low	Moderate	Medium	Composite conditions before enforcement
Rule mis-tuning causing missed real attacks (false negative)	Medium	Severe	High	Regular wazuh-logtest validation cycles

Appendix D — Glossary of Terms

Term	Definition
SIEM	Security Information and Event Management — a platform that aggregates, correlates, and alerts on log data from across an environment. Wazuh is the SIEM used throughout this internship.
FIM	File Integrity Monitoring — tracks changes (add/modify/delete) to specified files or directories and alerts when unexpected changes occur.
Syscheck	The name of Wazuh's internal FIM module, configured inside the <syscheck> block of ossec.conf.
Decoder	The Wazuh component that parses a raw log line into structured, named fields (e.g. srcip, dstuser) before rules can evaluate it.
Rule chaining (if_sid / if_matched_sid)	A mechanism for a custom rule to inherit the parsed fields and match conditions of a parent rule rather than re-parsing raw logs from scratch.
Active Response	A Wazuh framework that lets the Manager instruct an agent to execute a remediation script (e.g. a firewall block) automatically when a rule fires.
MITRE ATT&CK	A publicly maintained knowledge base of adversary tactics and techniques, used here to tag each custom rule with the behavior it detects.
IoC	Indicator of Compromise — an artifact (file hash, IP, log pattern) observed during an intrusion that suggests a system has been compromised.
Rule frequency / timeframe	Wazuh rule attributes that require an event pattern to repeat a set number of times (frequency) within a bounded window in seconds (timeframe) before firing — used to separate bursts from isolated events.
wazuh-logtest	A CLI/dashboard tool that runs a sample log line through the live decoder and rule set to show exactly which rule (if any) matches — used to debug and tune custom rules.

Appendix E — Personal Reflection & Skills Developed

E.1 Technical Skills Reinforced This Week

- Reading and hand-editing `ossec.conf` and `local_rules.xml` directly, rather than relying only on the dashboard UI — this builds the muscle memory a real SOC analyst needs when working against a manager over SSH with no GUI available.
- Using `wazuh-logtest` as a debugging tool rather than guessing at rule syntax — the SSH brute-force rule only worked after tracing the actual parent rule ID (5710) instead of assuming it from documentation.
- Distinguishing real telemetry from simulated/injected telemetry, and being explicit about which is which in a report — a habit that matters more once findings start feeding into decisions other people rely on.
- Working within genuine hardware constraints (16 GB RAM across three concurrent VMs) and documenting what was verified versus what was designed-but-not-executed, rather than overstating completion.

E.2 Where the Week Fell Short

Two gaps are worth naming plainly rather than glossing over. First, the Windows side of detection (Rule 100003, and the Event ID 4625/4720/4672 findings in Days 12–13) was never validated against a real Windows agent — it is decoder-pipeline testing at best, not incident detection. Second, the Day 14 Active Response chain was designed and mapped but not run end-to-end due to host resource limits, so the firewall-drop enforcement step is unverified in practice, not just in theory. Both gaps are flagged in-line above rather than left implicit, and both are the natural next steps once a Windows agent and more host resources are available.

Appendix F — Next Steps: Week 3 Planning

F.1 Immediate Follow-Ups From Week 2 Gaps

- Deploy an actual Windows 10/11 VM as a Wazuh agent to validate Rule 100003 and re-run the Day 12–13 Windows findings against real Security Log telemetry instead of logger-injected data.
- Re-attempt the Day 14 Active Response chain end-to-end (Hydra attack from Kali through to the iptables DROP rule) once host resources allow all three VMs to run concurrently without CPU starvation.
- Implement the `<white_list>` whitelisting safeguard identified in Section 6.2 before considering Active Response for anything resembling a production-style scenario.

F.2 Candidate Topics for Week 3

- Vulnerability detection module (Wazuh vulnerability-detector) and CVE correlation against installed packages.
- Log source expansion — bringing in a second Linux agent or a firewall/router log feed to practice multi-endpoint correlation.
- Formal incident response documentation practice — writing a full IR report against one of the simulated attack chains already built this week.

Section 7 — Consolidated Screenshot Ledger

Full index of supporting evidence files referenced throughout this report, in submission order.

#	File Name	Module	Description
1	W2_D8_syscheck_config_linux_01.PNG	Days 8–9	Active syscheck block in ossec.conf
2	W2_D8_agent_active_status_03.PNG	Days 8–9	ubuntu-agent Active status on Dashboard
3	W2_D9_FIM_all_alerts_linux.PNG	Days 8–9	FIM added/modified/deleted alert timeline
4	1_wazuh_rules_configuration.PNG	Days 10–11	local_rules.xml custom rule block
5	2_ubuntu_user_creation_command.PNG	Days 10–11	adduser command execution
6	3_ubuntu_ssh_brute_force_trigger.PNG	Days 10–11	SSH brute-force loop output
7	4_dashboard_alert_user_creation.PNG	Days 10–11	Rule 100001 alert on dashboard
8	5_dashboard_alert_ssh_brute_force.PNG	Days 10–11	Rule 100002 alert on dashboard
9	1_ubuntu_linux_log_generation.PNG	Days 12–13	Baseline SSH failure log generation
10	2_ubuntu_windows_event_injection.PNG	Days 12–13	Simulated Windows event logger injection
11	3_wazuh_manager_pipeline_sync.PNG	Days 12–13	wazuh-manager service restart
12	4_wazuh_dashboard_analysis_traces.PNG	Days 12–13	Ruleset Test decoder trace
13	5_wazuh_dashboard_csv_export_milestone.PNG	Days 12–13	Exported CSV in Notepad
14	Wazuh_Manager_Network_Interface_Verification.PNG	Day 14	Wazuh Manager eth0 verification
15	Kali_Linux_Attacker_Machine_Verification.PNG	Day 14	Kali attacker node interface verification
16	Ubuntu_Server_Endpoint_Configuration.PNG	Day 14	Ubuntu Server VirtualBox network adapter